

```
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv('IRIS.csv')
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df['species'].value_counts()
```

	count
species	
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

dtype: int64

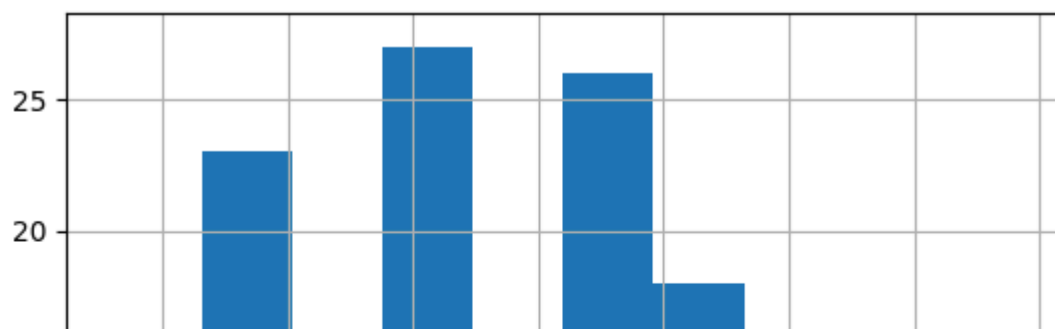
```
df.isnull().sum()
```

```
0
sepal_length 0
sepal_width   0
petal_length  0
petal_width   0
species       0
```

dtype: int64

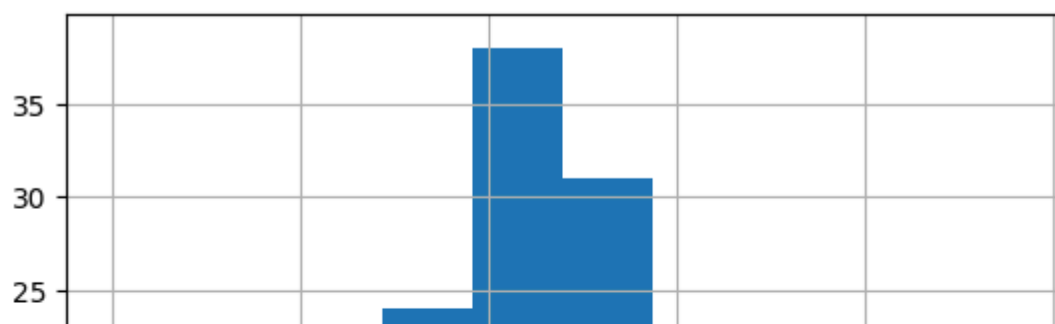
```
df['sepal_length'].hist()
```

<Axes: >



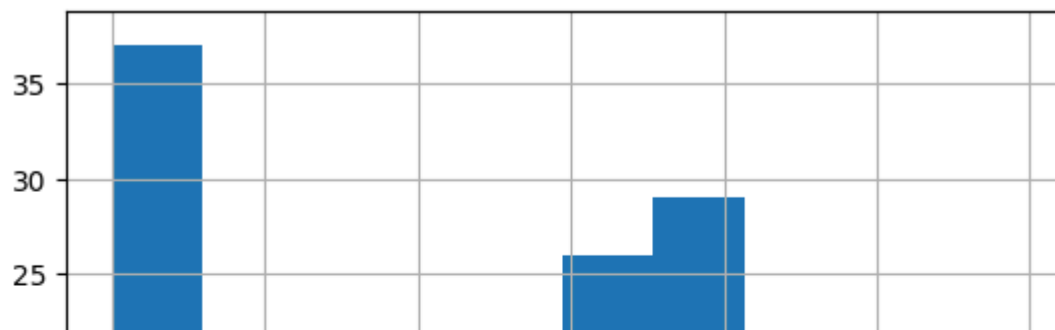
```
df['sepal_width'].hist()
```

<Axes: >



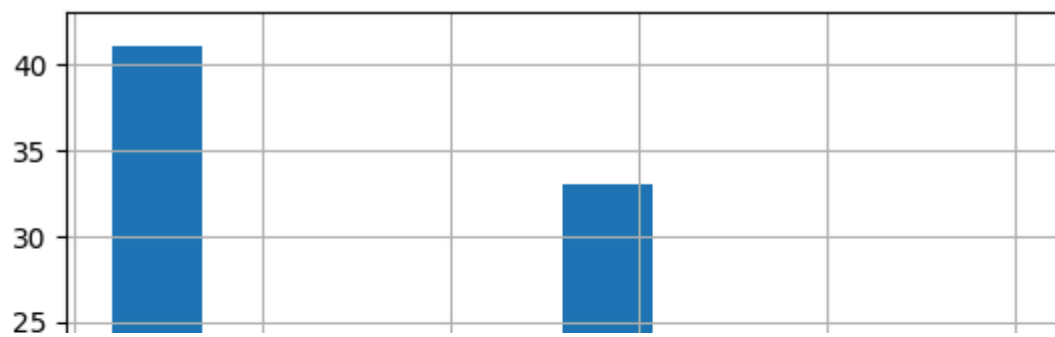
```
df['petal_length'].hist()
```

<Axes: >



```
df['petal_width'].hist()
```

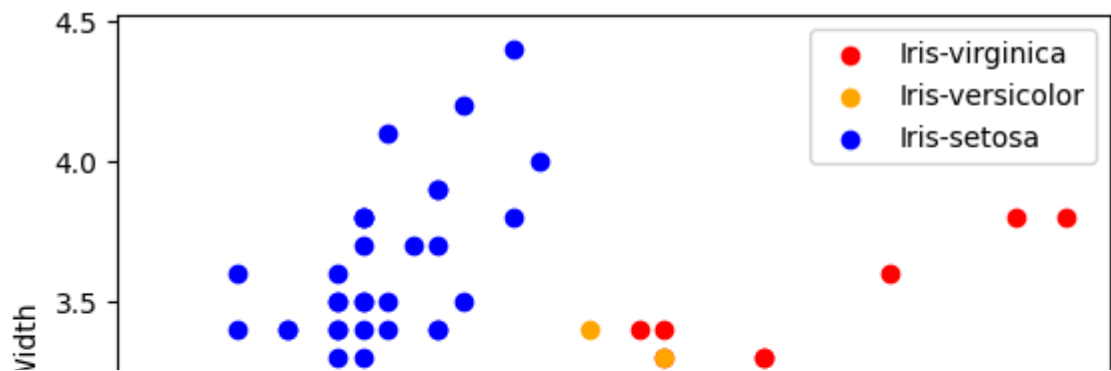
<Axes: >



```
colors = ['red', 'orange', 'blue']
species = ['Iris-virginica', 'Iris-versicolor', 'Iris-setosa']
```

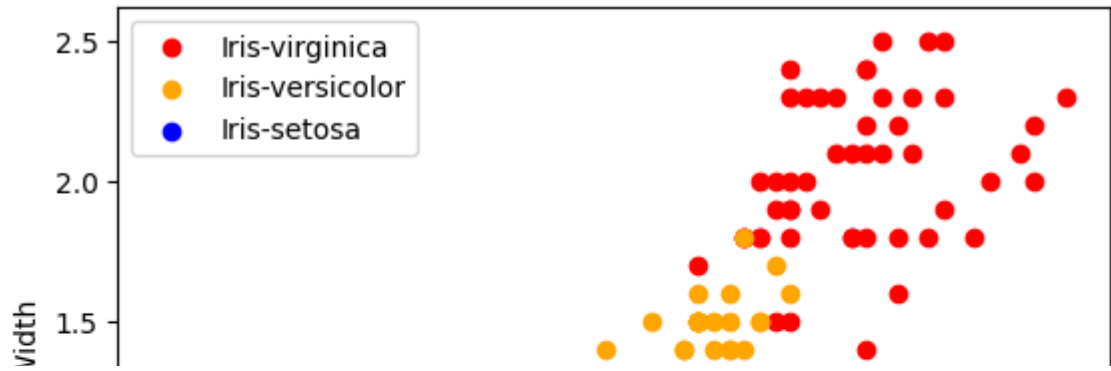
```
for i in range(3):
    x = df[df['species'] == species[i]]
    plt.scatter(x['sepal_length'], x['sepal_width'], c = colors[i], label=species[i])
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.legend()
```

<matplotlib.legend.Legend at 0x7b1c6845da10>



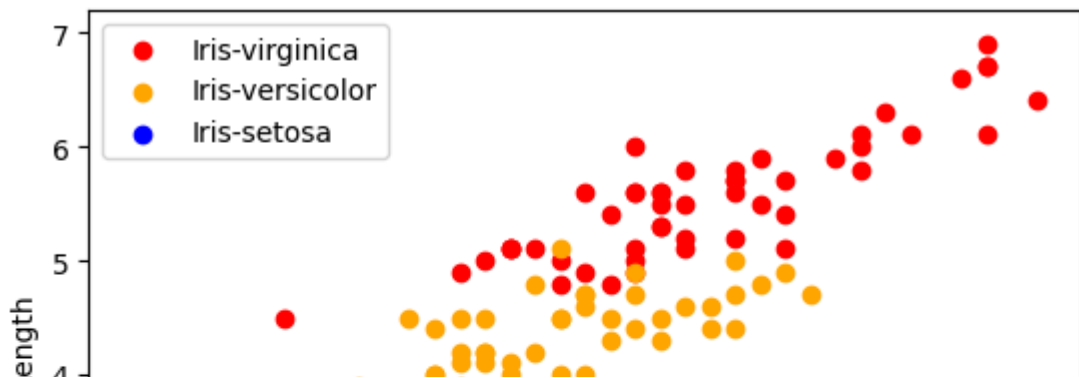
```
for i in range(3):
    x = df[df['species'] == species[i]]
    plt.scatter(x['petal_length'], x['petal_width'], c = colors[i], label=species[i])
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.legend()
```

<matplotlib.legend.Legend at 0x7b1c6a717a50>



```
for i in range(3):  
    x = df[df['species'] == species[i]]  
    plt.scatter(x['sepal_length'], x['petal_length'], c = colors[i], label=species[i])  
plt.xlabel("Sepal Length")  
plt.ylabel("Petal Length")  
plt.legend()
```

<matplotlib.legend.Legend at 0x7b1c684ada50>



```

for i in range(3):
    x = df[df['species'] == species[i]]
    plt.scatter(x['sepal_width'], x['petal_width'], c = colors[i], label=species[i])
plt.xlabel("Sepal Width")
plt.ylabel("Petal Width")
plt.legend()

```

<matplotlib.legend.Legend at 0x7b1c683e5a50>



```

df_numeric = df.select_dtypes(include=['number'])
correlation_matrix = df_numeric.corr()

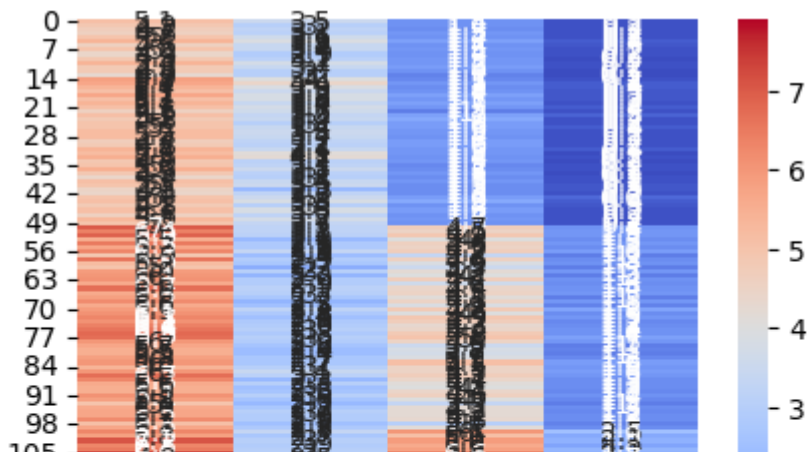
```

```

df_numeric = df.select_dtypes(include=['number'])
correlation_matrix = df_numeric.corr()
fig, ax = plt.subplots(figsize=(5,4))
sns.heatmap(df_numeric, annot=True, ax=ax, cmap = 'coolwarm')

```

<Axes: >



```
from sklearn.preprocessing import LabelEncoder  
le = LabelEncoder()
```

```
df['species'] = le.fit_transform(df['species'])  
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
from sklearn.model_selection import train_test_split  
# train - 70  
# test - 30  
X = df.drop(columns=['species'])  
Y = df['species']  
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.30)
```

```
from sklearn.linear_model import LogisticRegression  
model = LogisticRegression()
```

```
model.fit(x_train, y_train)
```

▾ LogisticRegression ⓘ ?

LogisticRegression()

```
print("Accuracy: ",model.score(x_test, y_test) * 100)
```

Accuracy: 100.0

```
from sklearn.tree import DecisionTreeClassifier  
model = DecisionTreeClassifier()
```

```
model.fit(x_train, y_train)
```

▾ DecisionTreeClassifier ⓘ ?

DecisionTreeClassifier()